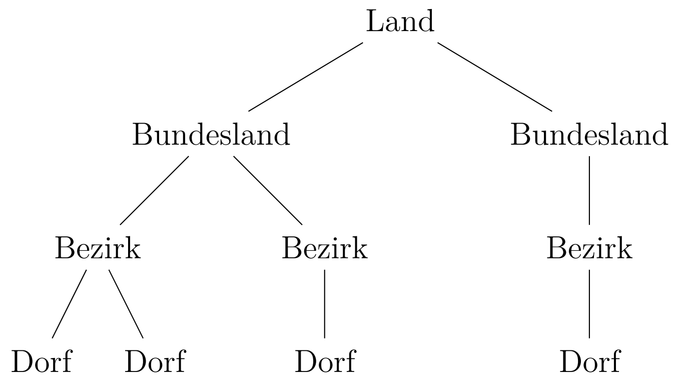
3-adischer Steuerbaum mit Blockchain-Hash und Zweckbindung

Alexander Kern

January 9, 2026

1 Hierarchie des Baums

3-adischer Baum mit Dörfern, Bezirken, Bundesländern und Ländern.



2 Blockchain-Hash

$$\text{hash}_{\text{node}} = H(\text{name}_{\text{node}} \parallel \text{parent\_hash} \parallel \Big\|_{c \in \text{children}} \text{hash}_c)$$

3 Steueraggregation mit Zweckbindung

$$\begin{aligned}
\text{tax}_{\text{dorf}} &= \text{tax}_{\text{infra}} + \text{tax}_{\text{transfer}} \\
\text{tax}_{\text{infra}} &= p_{\text{infra}} \cdot \text{tax}_{\text{dorf}} \\
\text{tax}_{\text{transfer}} &= (1 - p_{\text{infra}}) \cdot \text{tax}_{\text{dorf}} \\
\text{tax}^{\text{node}}_{\text{infra}} &= \text{tax}^{\text{local}}_{\text{infra}} + \sum_c \text{tax}^c_{\text{infra}} \\
\text{tax}^{\text{node}}_{\text{transfer}} &= \text{tax}^{\text{local}}_{\text{transfer}} + \sum_c \text{tax}^c_{\text{transfer}}
\end{aligned}$$

4 Algorithmus (Pseudocode)

```
Function GenerateNode(level, depth, parentHash):
    Name = generateName(level)
    if level == "Dorf":
        localTax = computeLocalTax()
        taxInfra = pInfra * localTax
        taxTransfer = (1-pInfra) * localTax
    else:
        children = []
        for 1..rand(1,3):
            child = GenerateNode(nextLevel(level), depth-1, parentHash)
            children.append(child)
        taxInfra = sum(child.taxInfra for child in children)
        taxTransfer = sum(child.taxTransfer for child in children)
    hash = SHA256(Name + parentHash + concat(child.hashes))
    return Node(Name, taxInfra, taxTransfer, hash, children)
```

5 Implementierungen

5.1 C++

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <openssl/sha.h>

struct Node {
    std::string name;
```

```
    double localTax = 0.0;
    double taxInfra = 0.0;
    double taxTransfer = 0.0;
    std::string hash;
    std::vector<Node> children;
};

double computeTax(Node& node){
    node.taxInfra = node.localTax*0.6;
    node.taxTransfer = node.localTax*0.4;
    for(auto& c : node.children){
        computeTax(c);
        node.taxInfra += c.taxInfra;
        node.taxTransfer += c.taxTransfer;
    }
    return node.taxInfra + node.taxTransfer;
}

void computeHash(Node& node, const std::string& parentHash=""){
    std::string data = node.name + parentHash;
    for(auto& c : node.children)
        data += c.hash;
    // SHA256 Funktion hier
    node.hash = "HASH_PLACEHOLDER";
    for(auto& c : node.children)
        computeHash(c, node.hash);
}
```

5.2 Java

```
class Node {
    String name;
    double localTax;
    double taxInfra;
    double taxTransfer;
    String hash;
    List<Node> children = new ArrayList<>();

    double computeTax(){
        taxInfra = 0.6*localTax;
        taxTransfer = 0.4*localTax;
        for(Node c : children){
```

```
            c.computeTax();
            taxInfra += c.taxInfra;
            taxTransfer += c.taxTransfer;
        }
        return taxInfra + taxTransfer;
    }

    void computeHash(String parentHash){
        String data = name + parentHash;
        for(Node c : children) data += c.hash;
        hash = "HASH_PLACEHOLDER";
        for(Node c : children) c.computeHash(hash);
    }
}
```

5.3 Haskell

```
data Node = Node {
    name :: String,
    localTax :: Double,
    taxInfra :: Double,
    taxTransfer :: Double,
    hashVal :: String,
    children :: [Node]
} deriving Show

computeTax :: Node -> Node
computeTax n =
    let infra = 0.6 * localTax n
        transfer = 0.4 * localTax n
        updatedChildren = map computeTax (children n)
        totalInfra = infra + sum (map taxInfra updatedChildren)
        totalTransfer = transfer + sum (map taxTransfer updatedChildren)
    in n { taxInfra = totalInfra, taxTransfer = totalTransfer, children = u

computeHash :: Node -> String -> Node
computeHash n parentHash =
    let childHashes = concatMap hashVal (children n)
        h = "HASH_PLACEHOLDER"
        updatedChildren = map (\c -> computeHash c h) (children n)
    in n { hashVal = h, children = updatedChildren }
```